Monopolizing Conversations: Examining Developer Conversation Participation in Chat Applications

Shane Ellis*
Nathan Stechschulte*
shaneellis@unomaha.edu
nstechschulte@unomaha.edu
University of Nebraska Omaha
Omaha, Nebraska, USA

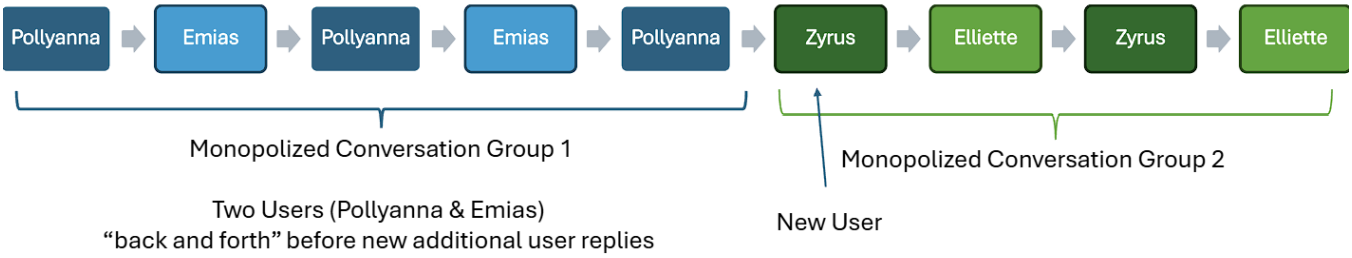

Figure 1: Analyzing Chat Conversation Patterns by Breaking Them into Conversations Between Groups of Users

Abstract

Problem: Software developers communicate through a wide array of online methods such as Discord, Slack, Microsoft Teams, Stack Overflow, Reddit, and more. Especially in the case of newer chat channels from applications such as Discord there is a different communication etiquette involving more frequent communication within individual channels. **Gap:** Prior research exists with respect to studying different facets of developer interaction through online chat mediums such as dataset compilation, experimentation, interviews, tools, and surveys but there is an absence of research for investigating monopolization characteristics found among conversations. **Aim:** Through our shared interest of expanding the collective knowledge surrounding developer communication we have provided a basis for further research to occur through using prior data to examine whether there is a detectable point in conversations to which the effects of monopolization are apparent. **Method:** We designed a tool to analyze conversation data for use with selected datasets to determine the likelihood of a person contributing to a conversation being an existing member or a new member. **Result:** We successfully built an application that can analyze Discord datasets and found that we do see a pattern in that the longer a conversation goes between a group of two individuals, the less likely someone else outside that group will reply. **Conclusion:** Our analysis indicated that longer conversations between two developers tend to discourage others from joining, which could reduce overall community participation and inclusivity.

1 Introduction

Software developers' capacity to communicate through digital means is an essential practice for continual learning opportunities, collaboration with other developers, and acquiring problem solving experience. Currently, there are many online communication routes available for developers to converse. Such is the case with forum oriented websites like Stack Overflow and Reddit, or chat applications such as Discord, Slack, and Microsoft Teams that are increasingly popular. With chat applications, there is still much to understand with respect to how developers conduct themselves on said applications. Interaction over chat applications is done in the form of shorter messages among users within chats. This style of communication can exhibit more frequent interwoven communication with less attention being paid to certain conversations.

There exists a variety of previous research instances contributing to an expanded understanding of developer interactions. Dataset collection [1, 11], experimentation [6], tool development [10], interviews [6, 7], and surveys [4, 6] of software developers were some of the methods found previously used. However, a gap that we observed in familiarizing ourselves with the broader subject matter was a lack of coverage related to conversation monopolization. We approached this research with the following research question in mind: **RQ - Do the number of posts present in a conversation effect prospective participants?**

For our contribution to studying developer communication, we sought to further explore the concept of monopolizing conversations, exemplified in Figure 1, through examining available data. Within our research we utilized an existing dataset titled DISCO [11] which contained developer conversation data from the Discord chat application. In working with said data we provided a starting point for continued research into conversation monopolization as

*Both authors contributed equally to this research.

well as an initial analysis into whether there is a detectable point at which monopolization begins to impact conversations. For conducting our analysis into conversation monopolization we developed an application [2] for processing input data for determining percentages regarding subsequent messages following a conversation, for a group size of two, for whether the message would occur from inside or outside a group.

Analysis conducted by our application provided results that allowed us to visualize a pattern in the data with respect to conversation length for the likelihood in which new individuals are to reply. The pattern among the data was more pronounced among larger developer communities with more conversations present, but similar observations were made from smaller communities captured in the selected dataset. From our results, we saw conversations of increasing message counts were less likely to receive input from new individuals.

Our contributions from the research are:

- The monopolizing-convo tool [2] for use in analyzing data for monopolization based on the number of messages in a conversation.
- An analysis of all developer conversations found within the DISCO dataset [11] for conversation monopolization.
- Discussion over outputs obtained by the monopolizing-convo tool.
- Opportunities for further research into studying conversation monopolization.

2 Related Work

A variety of research instances from recent years explored developer interactions seen across various online communication platforms. Chatterjee et al. [1] created a dataset housing developer conversations captured from three developer communities found on the Slack communication app during a two year time span and iterated on a prior algorithm for chat disentanglement [3] to address specific needs regarding larger monitored time segments between community messages. Building on work from Chatterjee et al. in a separate chat application Subash et al. [11] compiled a dataset featuring approximately one year worth of developer Discord conversations found across four active communities and selectively utilized a modified chat disentanglement algorithm [1, 3] on random conversations from among the larger harvested data pool for validation purposes. Lill et al. [6] carried out a mixed method analysis thorough experimentation, surveys, and interviews to study repeated questions found across four Discord communities with an accompanied approach. Ndukwe et al. [7] conducted a series of interviews to learn about developer relationships with question and answer websites. Sarker et al. [10] developed the toxicity detection tool ToxiCR for use in identifying instances of toxicity among developer comments found within free and open-source software projects. While many research instances have been carried out previously to study developer communication using online platforms, we did not explicitly observe research pertaining to the examination of conversation monopolization occurring in said settings which prompted our research for an increased knowledge on the matter.

3 Methodology

The following subsection outlines the methodology we employed for our research endeavor regarding examining developer participation within chat applications. Our methodology, provided the dataset of interest [11], encompassed the application development process in creating our simple tool for dataset analysis.

3.1 Tool Design

3.1.1 Basis of Tool. We elected to construct our own tool for data analysis with respect to our established research question exploring developer conversations as we were not able to encounter a specific tool that aligned with our interests. Simply named, the tool entitled **monopolizing-convo** may be found in our attached replication package [2] containing our source code data analysis output, application log, and paper documentation. The program created for analyzing the selected dataset of developer Discord conversations created by Subash et al. [11] operated through in taking conversations and performing analysis on grouped conversations. We elected to program the tool using the Java programming language due to both authors' shared familiarity with the language and one author's experience consisting of more than twelve years worth of professional experience working in the language. We sought to create the analysis tool using a modular programmatic structure that would allow for a standard format compatible with additional data inputs from other research instances.

Our monopolizing-convo tool leveraged the Extract, Transform, Load (ETL) process [5] as a part of accepting, processing, and analyzing data prior to logging and outputting results. The first step involved importing XML data found within the DISCO dataset [11] into our tool which was seen organized into folders for conversation data collected for each Discord developer community with sub-folders containing XML files for collected data from each month of their study. For easier processing by the tool further into use, we combined multiple months of collected developer conversations within each respective community into a single dataset file input for analysis as a singular instance. After importing the data, we created an XML schema with the assistance of ChatGPT [8] for defining the intended structure for XML data read in. The use of said schema helped in translating the XML data provided by the dataset into a Java object structure using Java Architecture for XML Bindings (JAXB) [9]. Figure 2 illustrates order of operations undertaken by the tool.

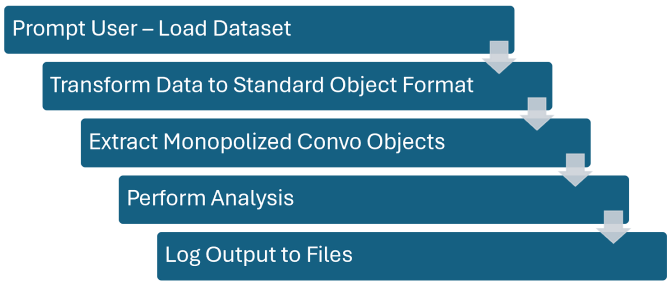

Figure 2: Application is built in a modular approach improve how easy it is to incorporate new data and analysis processes

3.1.2 Programmatic Approach. The procedure followed by the monopolizing-convo tool operated under the assumption of receiving an ordered series of messages translated from input data meaning the messages provided were delivered in sequence. Analysis on the resulting data structures created based upon input data was used alongside a "monopolized conversation" group size variable made configurable within the tool. By default, we configured the size of the grouping variable to focus attention on conversations made between two individuals while examining the occurrence of an additional member contributing to a conversation. If desired, the grouping size could be adjusted for viewing variations in analysis results such as incrementing the value to consider conversations between three people before a fourth joined. For this research instance, we directed our attention to analyzing conversations occurring between two participants with an interest regarding the likelihood of a third participant making a contribution.

During operation, monopolizing-convo built a list of messages in accordance with the configured variable for group size until a new unique participant contributed to a conversation. Concerning the nature of the research problem we explored, this enabled conversations to be grouped based on monopolization between a pairing of people. Our initial approach for analyzing conversations for monopolization centered around reviewing the probability of the next message in a discussion originating from someone previously apart of a conversation group or from someone new to the conversation group based on all measured conversations of a given message count.

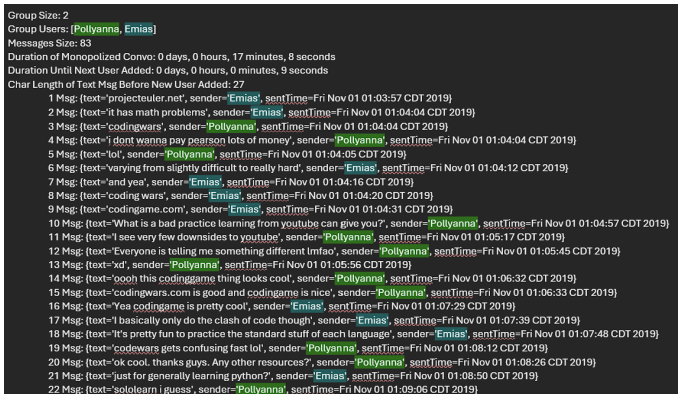

Figure 3: Example output of a monopolized conversation object. We see Pollyanna and Emias exchange 83 messages between only each other. Color formatting added post-processing for clarity.

4 Results

4.1 Output

We setup our program to run the entire DISCO dataset [11] for each Discord channel (Python, Racket, GoLang, and Clojurians). We found the program ran fast with only the output for the finalized metrics we looked to capture. That being: If the next message was found from within a given monopolized conversation group or was from a new user outside the monopolized conversation group. We setup the program to output examples of the monopolized conversations if a conversation went over a given length. In doing this, we procured samples of these longer conversation instances. We setup the program to output everything, but found significant bottlenecks in its performance due to the necessary operations involved in saving outputs to textual formats. With additional time, we could improve the program by incorporating a database or similar revision, but that would only necessary if we needed to manually review included datasets. Figure 3 shows an example of the output structure for each monopolized conversation captured. We attempted to capture other metrics such as the duration of the messages included within the example, the time elapsed until encountering a new user, the time accrued until the next conversation occurred, and the character length for the last message of a conversation prior to a new conversation beginning with the intention of being used in further studies.

4.2 Summarized Data

To summarize the data analysis conducted by the tool, we focused on the total number of messages for each developer community along with the number of messages found in each conversation grouping and percentages for next messages being inside or outside an observed group. The resulting analysis summarization shown in Table 1 allowed for viewing the four developer communities side by side. We grouped all messages with a length of fifteen messages or more into a singular row at the bottom of the table. This was done for conciseness within the report while still presenting findings from our analysis tool. The full results obtained from our analysis may be seen within our attached replication package [2] for more in-depth viewing purposes.

From Table 1 we see that the percentages relative to the Python developer Discord community possessed an inverse relationship in that the percentage of a subsequent message coming from a member within a group increased with the total number of messages in a conversation while the percentage from an individual not from a group decreased as message length increased. For the other three communities, the percentage likelihood of a message stemming from a member in a conversation group remained the dominant percentage shown across conversations with variable numbers of messages.

To help visualize the impact conversation length had in a monopolized conversation, we graphed the percentages for the next message following a conversation grouping coming from a participant found to be outside the conversation group. The graphical representation for each developer Discord community was represented as a separate line in Figure 4. Declines can be seen for each respective developer community. The overall declines seen for each developer community began as gradual declines with more frequent spikes in percentage measurements occurring as the total number of messages in a conversation increased. Percentage spikes for the Racket and Clojurians communities were found to occur sooner in the broader observation of conversation lengths whereas the spikes for the Python and GoLang communities increased more noticeably in frequency in longer conversations.

	Python			Racket			GoLang			Clojurians		
# of Msg in Group	Total # of Msg	% Next Msg Within Group	% Next Msg Outside Group	Total # of Msg	% Next Msg Within Group	% Next Msg Outside Group	Total # of Msg	% Next Msg Within Group	% Next Msg Outside Group	Total # of Msg	% Next Msg Within Group	% Next Msg Outside Group
2	342079	37%	63%	497	69%	31%	30850	64%	36%	119	67%	33%
3	153730	39%	61%	369	59%	41%	23555	60%	40%	99	71%	29%
4	72663	43%	57%	278	69%	31%	16746	61%	39%	83	61%	39%
5	38101	49%	51%	203	64%	36%	11928	62%	38%	57	67%	33%
6	22689	54%	46%	169	70%	30%	8694	65%	35%	43	74%	26%
7	14428	57%	43%	129	61%	39%	6502	67%	33%	35	77%	23%
8	9796	62%	38%	93	73%	27%	5029	69%	31%	31	77%	23%
9	7025	64%	36%	72	75%	25%	3923	72%	28%	26	77%	23%
10	5268	66%	34%	61	82%	18%	3208	74%	26%	21	86%	14%
11	4092	67%	33%	57	75%	25%	2649	74%	26%	20	85%	15%
12	3090	69%	31%	44	82%	18%	2177	77%	23%	17	88%	12%
13	2482	71%	29%	47	74%	26%	1910	77%	23%	17	88%	12%
14	1980	72%	28%	26	92%	8%	1599	77%	23%	14	93%	7%
15+	12523	79%	21%	602	91%	9%	14649	84%	16%	245	89%	11%

Table 1: Analysis results of the DISCO dataset [11] using our application analyze monopolizing conversations

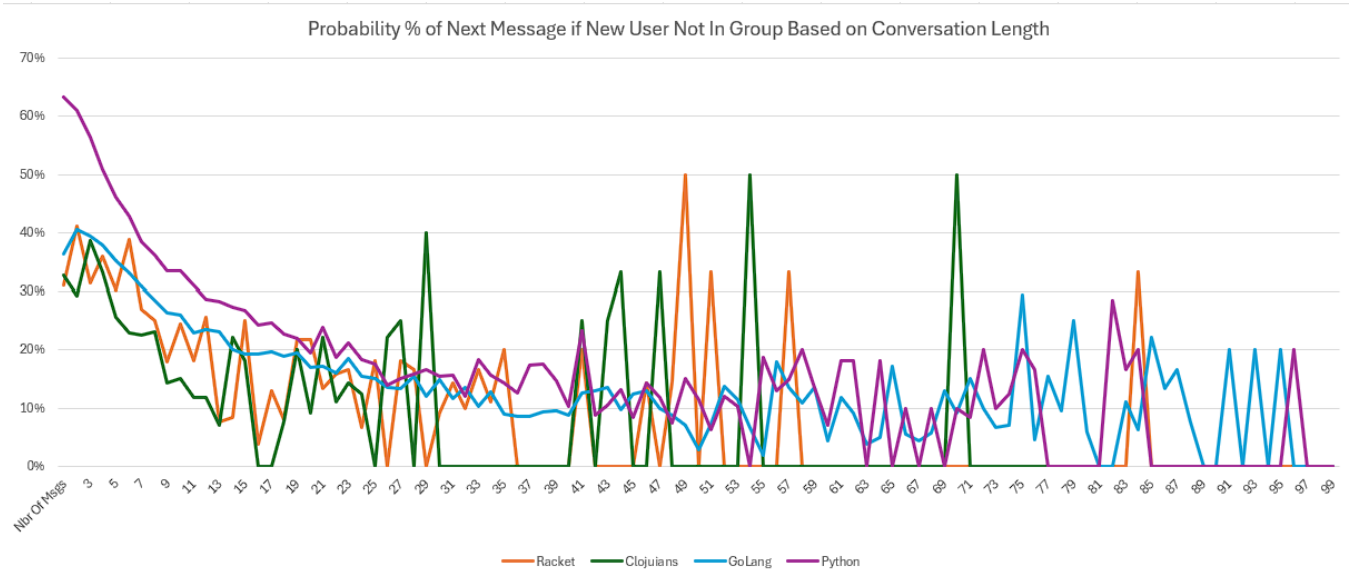

Figure 4: For each Discord channel, we see a consistent pattern of reduced probability of the next message being someone outside the group. The spikes are due to low sample size.

5 Discussion

5.1 Observations

From the largest community captured in the selected dataset [11], that being the Python community, we could see a clear trend for conversation monopolization involving groups of two represented in both Table 1 and Figure 4. We saw that as a conversation increased in message count, the probability percentage of a new user found not to be in a given conversation group decreased steadily in favor of those already found to be within a given discussion circle. Interestingly, for conversations with six or more messages present, the majority percentage shifted in favor of those found to be in the conversation with respect to who the next message would be from while conversations with fewer messages present posed percentages in favor of individuals outside a conversation providing the next message.

We saw similar trends from the remaining three communities for Racket, GoLang, and Clojurians, but none of those communities possessed as a pronounced percentage relationship as seen with the Python community. The size of the latter three Discord

communities could have been a contributing factor to the more subtle relationships observed between percentages for new and existing users for the communities. Looking to Figure 4, presence of percentage spikes for messages provided from outside individuals spiking sooner in conversation lengths and more frequently which could also look to be attributed to smaller community sizes and total message counts for conversations of differing sizes seen in each community. Further research would be necessary for validating the statistical significance of the behaviors seen across the Discord communities, especially the three smaller communities. The additional research would also be to explore other potential contributing factors encountered for this research outing.

5.2 Future Work

With respect to future studies, several possibilities for the continuation of study into monopolizing conversations have been thought over. One consideration to further explore would be the duration of time between messages in a group and whether that would present an observable impact on new replies taking place. Additional exploration into message size and associated impact would also be of interest in examining for monopolization. Previous studies with developer chat applications looked into average message length with developer communities [1] but were not focused on monopolization specifically. A behavior observed through this initial research was the tendency for individuals to repeat messages, or in some cases, spam messages repeatedly. Lill et al. [6] studied repeated questions over Discord with an automated approach for identifying said types of messages. In the case of monopolization, insight could be attained through investigating repeated messages as well. Gathering a stronger understanding of the context of message content could also expand understanding of conversation monopolization. Potential for toxicity found in messages [10] or greater conversational participation and content evocative of Impostor Phenomenon [4] could also impact prospective participants.

6 Limitations

6.1 Internal Validity

We noted as we continued our research that more exploratory research would be beneficial for statistical findings to be validated. We brainstormed many ideas for future work that could indicate alternative reasons why individuals outside a group may not reply. We noted how we built our application to dynamically change the group size, but with the time available ran the datasets for a group size of two. We do want to note this ultimately was not a concern because our goal was not to prove anything with this limited scope but rather build an expandable tool and try to gain insight with our prediction in that the longer a monopolized conversation length was the less probable someone new outside the group would reply.

6.2 External Validity

We saw that the results somewhat varied between our four Discord servers provided in the DISCO dataset [11]. There was some concern that these results might vary based on other factors outside the conversation length, such as the attributes of the server (like server population size, moderation policies, and hosting platform). However, we noted that we analyzed not just a subset of each dataset, but the entire population that included over 1.5 million messages [11] processed.

6.3 Construct Validity

We acknowledged the possible presence of a mono-method bias seeing as we only used a single method and data source [11] to construct our tool and analyze the data. To attempt to mitigate this, we did build the program in a way that would support more datasets with limited changes. However, due to time constraints set forth we were unable to process additional datasets. As mitigation we noted the DISCO dataset [11] was also one of the larger reputable datasets available and we analyzed it in its completeness.

7 Conclusion

Our study explored the concept of monopolization conversations in developer chat applications by analyzing how conversation length between individuals affected the likelihood of others joining the discussion. Using our custom-built analysis tool on the DISCO dataset, we observed a consistent trend: As conversations between two individuals grew longer, the likelihood of new participants contributing decreased significantly. This pattern was most evident in larger communities like Python, though similar trends were seen across smaller ones. These findings showed that long conversations between two people can unintentionally make others less likely to join in, which may affect how inclusive and active the community is. We hope this work encourages further exploration into the dynamics of developer communication as well as the design of tools and norms that promote more inclusive interaction.

References

[1] Preetha Chatterjee, Kostadin Damevski, Nicholas A. Kraft, and Lori Pollock. 2020. Software-related Slack Chats with Disentangled Conversations. In *Proceedings of the 17th International Conference on Mining Software Repositories* (Seoul, Republic of Korea) *(MSR '20)*. Association for Computing Machinery, New York, NY, USA, 588–592. doi:10.1145/3379597.3387493

[2] Shane Ellis and Nathan Stechschulte. 2025. *monopolizing-convo: Replication Package.* Retrieved May 4, 2025 from https://github.com/nsteck17/monopolizing-convo

[3] Micha Elsner and Eugene Charniak. 2008. You Talking to Me? A Corpus and Algorithm for Conversation Disentanglement. In *Proceedings of ACL-08: HLT*, Johanna D. Moore, Simone Teufel, James Allan, and Sadaoki Furui (Eds.). Association for Computational Linguistics, Columbus, Ohio, 834–842. https://aclanthology.org/P08-1095/

[4] Paloma Guenes, Rafael Tomaz, Marcos Kalinowski, Maria Teresa Baldassarre, and Margaret-Anne Storey. 2024. Impostor Phenomenon in Software Engineers. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society* (Lisbon, Portugal) *(ICSE-SEIS'24)*. Association for Computing Machinery, New York, NY, USA, 96–106. doi:10.1145/3639475.3640114

[5] IBM. 2021. *What is ETL (extract, transform, load)?* https://www.ibm.com/think/topics/etl

[6] Alexander Lill, André N. Meyer, and Thomas Fritz. 2024. On the Helpfulness of Answering Developer Questions on Discord with Similar Conversations and Posts from the Past. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 58, 13 pages. doi:10.1145/3597503.3623341

[7] Ifeanyi G. Ndukwe, Sherlock A. Licorish, and Stephen G. MacDonell. 2023. Perceptions on the Utility of Community Question and Answer Websites Like Stack Overflow to Software Developers. *IEEE Transactions on Software Engineering* 49, 4 (2023), 2413–2425. doi:10.1109/TSE.2022.3220236

[8] OpenAI. 2025. *ChatGPT*. https://chatgpt.com

[9] Ed Ort and Bhakti Mehta. 2003. *Java Archtecture for XML Binding (JAXB).* https://www.oracle.com/technical-resources/articles/javase/jaxb.html

[10] Jaydeb Sarker, Asif Kamal Turzo, Ming Dong, and Amiangshu Bosu. 2023. Automated Identification of Toxic Code Reviews Using ToxiCR. *ACM Trans. Softw. Eng. Methodol.* 32, 5, Article 118 (July 2023), 32 pages. doi:10.1145/3583562

[11] Keerthana Muthu Subash, Lakshmi Prasanna Kumar, Sri Lakshmi Vadlamani, Preetha Chatterjee, and Olga Baysal. 2022. DISCO: a dataset of discord chat conversations for software engineering research. In *Proceedings of the 19th International Conference on Mining Software Repositories* (Pittsburgh, Pennsylvania) *(MSR '22)*. Association for Computing Machinery, New York, NY, USA, 227–231. doi:10.1145/3524842.3528018